ExamForge AI

Performance Report

Build performance, bundle analysis, and optimization assessment

Generated: August 02, 2026 at 01:06 UTC

Confidential

1. Build Performance

The production build compiles successfully in approximately 22 seconds using Turbopack. The build generates 40 routes with a mix of static and dynamic pages. Server components are used extensively for data-fetching pages, which reduces client-side JavaScript bundle size. Client components are used only where interactivity is required (forms, charts, realtime subscriptions).

2. Server Component Strategy

The application leverages Next.js 16 App Router with React 19 Server Components for the majority of data-fetching pages. The following pages are server components that render on the server with zero client-side JavaScript: Schools, Students, Teachers, Parents, CBT/Exams, Results, Question Bank, Marketplace, and Billing. This approach significantly reduces the client-side bundle size and improves initial page load performance.

3. Dynamic Rendering

All server components that use Supabase auth are configured with force-dynamic rendering, ensuring fresh data on every request. This is critical for authenticated pages where data changes frequently. Static pages (login, register, forgot-password, etc.) are pre-rendered at build time for optimal performance.

4. Route Analysis

Route	Render Mode	Component Type	Description
/	Static	Server	Landing page
/login	Static	Client	Auth form
/register	Static	Client	Auth form
/dashboard/*	Dynamic	Server	Role-based stats
/schools	Dynamic	Server	DataTable with filters
/students	Dynamic	Server	DataTable with avatars
/teachers	Dynamic	Server	DataTable with filters
/parents	Dynamic	Server	DataTable with children
/analytics	Static	Client	Recharts visualizations
/cbt	Dynamic	Server	DataTable with tabs
/results	Dynamic	Server	DataTable with progress bars
/question-bank	Dynamic	Server	DataTable with filters
/marketplace	Dynamic	Server	Product cards with tabs
/billing	Dynamic	Server	Plan info, usage, invoices

Route	Render Mode	Component Type	Description
/reports	Static	Client	Report tables with export
/search	Static	Client	Global search across entities
/notifications	Static	Client	Realtime subscription

5. Key Libraries

The application uses a carefully selected set of libraries optimized for performance: Recharts for chart rendering (lightweight, SVG-based), TanStack Table for data tables (virtualized, efficient), Zustand for state management (minimal bundle size), and Supabase JS client for realtime and data fetching. The shadcn/ui component library provides tree-shakeable components that only include what is used.

6. Optimization Opportunities

Future performance optimizations include: implementing dynamic imports for heavy chart components, adding Suspense boundaries for streaming data loading, implementing ISR for semi-static pages like Marketplace, optimizing images with next/image, and adding React.memo for expensive re-renders in client components. These are recommended but not blocking for production launch.